

ToSh Cheat Sheet: Core Syntax

Expressions, variables, operators, and functions

parser/tests first

Literals And Strings

```
# numbers
42 3.14 1_000_000 0xFF 0b1010 0o777

# strings
"escapes\nok"
'raw text'
"""multi-line"""
$"hello {$name}"
$'ANSI C escapes: \x1b[31m'

# specials
true false null
2026-03-25
127.0.0.1
```

Form	Notes
"..."	Escapes, no interpolation
'...'	Raw, no interpolation
"""..."""	Raw multi-line block
`\${}`	Interpolation with {...}
\$'...'	ANSI-C escape processing

Variables And Assignment

```
var name = "Alice"
var age: int = 30
$age = ($age + 1)
$name ??= "guest"

var [first, second] = [10, 20, 30]
var { Name, Uid } = {| Name = "toast", Uid = 1000 |}

var base = {| Name = "Alice", Role = "user" |}
var admin = {| ...$base, Role = "admin" |}
```

Variables must be declared with `var` before use. Assignment targets use the `$name` form. Records use paired `{| ... |}` delimiters; plain braces remain blocks and destructuring patterns.

Operators

Operator	What it covers
+ - * / %	numeric math, date arithmetic, string/list concat
== != < > <= >=	equality and ordered comparison
= !	regex match and negated regex match
and or not	boolean logic; && and also work
is is-not as	type check and cast-style conversion
in not in	membership testing
?? ?. ??=	null handling
?:	ternary conditional

```
echo ("hello" =~ "^h")
echo (42 is int)
echo ("a" in ["a", "b", "c"])
var label = $count > 0 ? "items" : "empty"
```

Functions

```
func greet(name) {
    writeline $"Hello, {$name}!"
}

func double(x) => $x * 2
func add => $1 + $2

func connect(host: string, port: int) { ... }
func greet(name: string, title?: string = "friend") { ... }
func print-all(items...) { for item in $items { echo $item } }
func add(a: int, b: int) -> int => ($a + $b)

var fn = func(x) => $x * 2
invoke $fn 21
var ref = &double
[1, 2, 3] | map &double
```

See also: the control-flow sheet covers `if`, `loops`, `try/catch/finally`, `defer`, `switch`, and `match`. A natural ToSh style is typed parameters for public helpers, block syntax for pipeline predicates, and interpolated strings for readable output.

Comments And Identifiers

```
# one-line comment
echo hello # inline comment

var my_count = 10
var total-items = 42
func process-batch() { ... }
```

- ToSh supports single-line comments only.
- Hyphens are allowed in identifiers, which keeps shell-style names natural.
- Reserved words like `func`, `class`, and `match` cannot be rebound.

More String Forms

Form	Use
<code>"'...'"</code>	raw multi-line block
<code>\$"..."</code>	interpolated multi-line block
<code>\$"'"..."'</code>	ANSI-C escaped block text

```
var raw = '''line 1
line 2'''

var msg = $"User: { $name }
Count: { $count }"
```

Ranges

```
1..10
1..2..10
10..1

for i in 1..5 {
    echo $i
}
```

- Ranges are lazy shell values, not eagerly materialized arrays.
- They iterate naturally in `for` loops and can be splatted into commands.

Null And Assignment

```
var name = $user ?? "Anonymous"
var len = $user?.Name?.Length ?? 0

var x = 10
$x += 5
$x *= 2
$x ??= 100
```

Operator	Meaning
<code>??</code>	fallback when left is null
<code>?.</code>	null-conditional member access
<code>??=</code>	assign only when target is null
<code>+= -= *= /=</code>	compound assignment

A useful parsing rule: ternary binds more loosely than `??`, so `null ?? true ? yes : no` reads like `(null ?? true) ? yes : no`.

Destructuring And Spread

```
var person = { | name = "Alice", age = 30 | }
var { name, age } = $person

var items = [10, 20, 30, 40]
var [first, second] = $items
var [_, _, third] = $items

var a = [1, 2, 3]
var b = [0, ...$a, 4]

var base = { | role = "user" | }
var admin = { | ...$base, role = "admin" | }

dotnet ...$publish_args
```

Computed Keys

```
var key = "name"
var record = { | ($key) = "Alice", age = 30 | }
echo $record.name
```

Callable Values

```
var fn = func(x) => $x * 2
invoke (func(x) => ($x * 2)) 21
var callback = &greet
[1, 2, 3] | map &double

func t1 => test1 $1 "Jim" $2

func upper_input() {
    $tosh.Function.Input | each { _.ToUpper() }
}
```

- `$1`, `$2`, ... are positional parameters in wrapper-style arrow functions.
- `$tosh.Function.Input` exposes piped input inside a function body.
- `$tosh.Function.Args` exposes the call argument list.

Member Access

```
"hello".ToUpper()
String.Join(" + ", ["a", "b"])
$person.Name
$person?.Manager?.Name
```

- Dot syntax is the preferred style for instance and static member access.
- Use `call` or `call-method` only when the invocation has to stay dynamic.

Style Hints

- Use interpolated strings for user-facing output and plain quoted strings for data.
- Use typed parameters on reusable public functions, but keep local helpers lightweight.
- Reach for destructuring and spread when they make data re-shaping shorter than repeated field-by-field assignments.